

Hierarchical Cyberbullying Detection Using a Two-Stage Deep Learning Architecture

Technical Documentation

1. Project Overview

ShieldGuard is an end-to-end real-time cyberbullying detection system built using a **Two-Stage Cascaded RoBERTa architecture**. It consists of:

- A **Flask web application** (`web_dashboard_app.py`) serving real-time predictions, an analytics dashboard, and an Explainable AI interface.
- A **Chrome browser extension** (`browser_extension/`) that scans social media posts automatically.
- An **evaluation pipeline** (`evaluate_pipeline.py`) for offline model benchmarking.

The AI core uses two fine-tuned `roberta-base` models with **Multi-Sample Dropout** for robust predictions:

- **Stage 1 (Binary):** Detects whether text is cyberbullying or not.
- **Stage 2 (Multi-class):** Categorizes the type of cyberbullying.

2. Software & Libraries Used

2.1 Python Dependencies (`requirements.txt`)

Library	Version	Purpose
<code>`flask`</code>	3.0.0	Web framework — serves the dashboard and REST API
<code>`flask-cors`</code>	4.0.0	Cross-Origin Resource Sharing for browser extension API calls
<code>`torch`</code>	$\geq 2.2.0$	PyTorch — model inference engine
<code>`transformers`</code>	$\geq 4.35.0$	HuggingFace — loads RoBERTa tokenizer and model weights
<code>`numpy`</code>	$\geq 1.26.0$	Numerical operations
<code>`unicorn`</code>	21.2.0	Production WSGI server (Linux/Mac deployment)

2.2 Additional Python Standard Library Modules

Module	Purpose
<code>`flask_limiter`</code>	Rate limiting (2000/day, 500/min)
<code>`logging`</code>	Application-level structured logging to <code>`app_errors.log`</code>
<code>`html`</code>	XSS prevention via <code>`html.escape()`</code> on user input
<code>`re`</code>	Regular expression for keyword matching in Explainable AI
<code>`collections.defaultdict`</code>	Aggregating statistics by label type
<code>`os`, `json`, `datetime`</code>	Standard file, JSON, and time utilities

2.3 Browser Extension (JavaScript)

Technology	Purpose
Chrome Extension Manifest V3	Extension specification and permissions
<code>`chrome.storage.local`</code>	Persisting user settings and scan statistics
<code>`chrome.runtime`</code>	Background service worker communication
<code>`chrome.tabs`</code>	Injecting content scripts and sending messages
Vanilla JavaScript	All popup and content script logic
CSS (Vanilla)	<code>`content_styles.css`</code> for injected UI overlays

2.4 Tools & Environment

Tool	Purpose
Python 3.10+	Runtime environment
CUDA (optional)	GPU acceleration — automatically detected via <code>`torch.cuda.is_available()`</code>
Google Chrome	Required to load the browser extension
pip / venv	Python package management

3. Project File Structure

CyberbullyingDetection_FinalYear/

- web_dashboard_app.py # Flask backend – main entry point
- evaluate_pipeline.py # Offline multi-threshold evaluation script
- requirements.txt # Python package dependencies
- models/
 - stage1_binary_model/ # Fine-tuned Stage 1 RoBERTa (binary)
 - best_model.bin # Trained weights (~475 MB)
 - tokenizer.json
 - tokenizer_config.json
 - vocab.json
 - merges.txt
 - special_tokens_map.json
 - stage2_multiclass_model/ # Fine-tuned Stage 2 RoBERTa (5-class)
 - best_model.bin # Trained weights (~475 MB)
 - tokenizer.json
 - ... (same tokenizer files)
- templates/ # Flask HTML templates (Jinja2)
 - index.html # Main analysis page
 - dashboard.html # Analytics dashboard
 - explain.html # Explainable AI page
 - youtube.html # YouTube integration page
- static/ # CSS, JS, images for the web app
- browser_extension/
 - manifest.json # Chrome extension package
 - background.js # Extension manifest (MV3)
 - content_script.js # Service worker – health checks every 5s
 - # Injects detection UI into social media
- pages
 - content_styles.css # Styles for injected overlays
 - popup.html # Extension popup UI
 - popup.js # Popup logic and settings management
 - icon*.png # Extension icons (16, 32, 48, 128px)
- english_andrewmvd_twitter/
 - cyberbullying_tweets.csv # Training/evaluation dataset

4. Core Source Code

4.1 Flask Web Application — `web\_dashboard\_app.py`

This is the main backend server.

Model Architecture (*`MultiSampleDropoutClassifier`*)

```
class MultiSampleDropoutClassifier(nn.Module):
    def __init__(self, model_name, num_labels, num_samples=3, dropout=0.3):
        super(MultiSampleDropoutClassifier, self).__init__()
        config = AutoConfig.from_pretrained(model_name)
        self.roberta = AutoModel.from_pretrained(model_name, config=config)
        self.dropouts = nn.ModuleList([nn.Dropout(dropout) for _ in
range(num_samples)])
        self.classifier = nn.Linear(config.hidden_size, num_labels)

    def forward(self, input_ids, attention_mask):
        outputs = self.roberta(input_ids=input_ids,
attention_mask=attention_mask)
        pooled_output = outputs.last_hidden_state[:, 0, :]

        logits_list = []
        for dropout in self.dropouts:
            dropped = dropout(pooled_output)
            logits = self.classifier(dropped)
            logits_list.append(logits)

        logits = torch.stack(logits_list, dim=0).mean(dim=0)
        return logits
```

How it works: The model applies 3 independent dropout layers to the [CLS] token representation and averages their outputs. This **Multi-Sample Dropout** technique reduces overfitting and improves generalization.

Key Configuration

```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
MAX_LENGTH = 128
DEFAULT_THRESHOLD = 0.70 # Toxic confidence threshold for Stage 1

STAGE2_LABELS = {
    0: 'Age-based',
    1: 'Ethnicity-based',
    2: 'Gender-based',
    3: 'Religion-based',
    4: 'Other Cyberbullying'
}
```

Two-Stage Prediction Logic (*`predict\_cyberbullying`*)

```
def predict_cyberbullying(text, threshold=DEFAULT_THRESHOLD):
    # Tokenize input
    encoding = tokenizer(
        text,
        add_special_tokens=True,
        max_length=MAX_LENGTH,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    ).to(DEVICE)
```

```

# Stage 1: Binary Classification
with torch.no_grad():
    logits_s1 = model_s1(encoding['input_ids'], encoding['attention_mask'])
    probs_s1 = torch.softmax(logits_s1, dim=1)[0]
    safe_conf = probs_s1[0].item()
    toxic_conf = probs_s1[1].item()

    if toxic_conf < threshold:
        return {'is_cyberbullying': False, 'label': 'Not Cyberbullying', ...}

# Stage 2: Multi-class Type Classification
with torch.no_grad():
    logits_s2 = model_s2(encoding['input_ids'], encoding['attention_mask'])
    probs_s2 = torch.softmax(logits_s2, dim=1)[0]
    pred_id = torch.argmax(probs_s2).item()

    label = STAGE2_LABELS[pred_id]
    combined_conf = toxic_conf * 0.7 + type_conf * 0.3    # Weighted confidence
    return {'is_cyberbullying': True, 'label': label, 'confidence':
combined_conf, ...}

```

API Endpoints

Method	Endpoint	Description
GET	/	Main analysis page
GET	/dashboard	Statistics dashboard
GET	/explain	Explainable AI page
GET	/youtube	YouTube integration page
POST	/api/predict or /api/analyze	Run cyberbullying prediction on text
POST	/api/explain	Get word-level explanation for a prediction
POST	/api/save-analysis	Save extension scan to history (auth required)
GET	/api/stats	Aggregate statistics on scanned content
GET	/api/history	Full scan history (up to 200 items)
POST	/api/clear-history	Clear all history
GET	/api/health	Health check for extension connectivity

Security Features

Feature	Implementation
Rate Limiting	2000/day, 500/min (Flask-Limiter)
CORS Restriction	Configurable via ALLOWED_ORIGINS env var
Secret Key	Loaded from SECRET_KEY env var, never hardcoded
Extension Auth	X-Extension-Secret header validated on /api/save-analysis
XSS Prevention	html.escape() on all user input before storing
Input Validation	Min 3 chars, max 2000 chars enforced on API

4.2 Chrome Browser Extension

`manifest.json` — Extension Configuration

```
{
  "manifest_version": 3,
  "name": "Cyberbullying Detector",
  "version": "1.0.0",
  "permissions": ["storage", "activeTab"],
  "host_permissions": [
    "http://localhost:5000/*",
    "*/twitter.com/*", "*/x.com/*",
    "*/facebook.com/*", "*/www.facebook.com/*",
    "*/reddit.com/*", "*/www.reddit.com/*"
  ],
  "background": { "service_worker": "background.js" },
  "content_scripts": [{
    "matches": ["*/twitter.com/*", "*/x.com/*", "*/facebook.com/*",
      "*/www.facebook.com/*", "*/reddit.com/*",
    "*/www.reddit.com/*"],
    "js": ["content_script.js"],
    "css": ["content_styles.css"],
    "run_at": "document_idle"
  }]
}
```

Supported Platforms: Twitter/X, Facebook, Reddit.

`background.js` — Service Worker

Performs a health check against `http://localhost:5000/api/health` every **5 seconds** and stores the result (online / offline) in `chrome.storage.local`. The popup reads this to show the connection status in real time.

```
async function checkHealth() {
  try {
    const response = await fetch('http://localhost:5000/api/health');
    if (response.ok) {
      await chrome.storage.local.set({ serverStatus: 'online' });
    } else { throw new Error('Server error'); }
  } catch (error) {
    await chrome.storage.local.set({ serverStatus: 'offline' });
  }
}
checkHealth();
setInterval(checkHealth, 5000);
```

`popup.js` — User Settings

Manages user-facing settings stored in `chrome.storage.local`:

- ``autoHide`` — Auto-hide detected cyberbullying posts
- ``showWarnings`` — Show warning overlays on flagged content
- ``sensitivity`` — Detection threshold slider (default: 70)
- Dashboard redirect button, manual page scan button, reset stats button.

`content\_script.js`

Injected into social media pages. Scans post text, calls the Flask `/api/predict` endpoint, and overlays warning badges on flagged content.

5. Instructions to Execute the Code

5.1 Prerequisites

Requirement	Details
Python	3.10 or higher
pip	Latest version
CUDA (optional)	NVIDIA GPU + CUDA toolkit for faster inference
Google Chrome	For the browser extension
Disk Space	~1 GB for the two model `.bin` files

5.2 Step 1 — Set Up Python Environment

Open a terminal (Command Prompt or PowerShell) inside the project folder:

```
# Navigate to the project directory
cd D:\CyberbullyingDetection_FinalYear

# Create a virtual environment
python -m venv venv_final

# Activate the virtual environment (Windows)
venv_final\Scripts\activate

# Install all required packages
pip install -r requirements.txt

# Also install evaluation pipeline dependencies
pip install pandas scikit-learn seaborn matplotlib tqdm
**Note:** If you have a CUDA-compatible GPU, install the GPU version of PyTorch:

``bash

pip install torch --index-url https://download.pytorch.org/whl/cu121

``
```

5.3 Step 2 — Verify Model Files

Ensure both trained model folders exist with the following files:

```
models/
├── stage1_binary_model/
│   ├── best_model.bin      ✓ Required (~475 MB)
│   ├── tokenizer.json      ✓ Required
│   ├── vocab.json          ✓ Required
│   └── merges.txt          ✓ Required
├── stage2_multiclass_model/
│   ├── best_model.bin      ✓ Required (~475 MB)
│   ├── tokenizer.json      ✓ Required
│   └── ...
```

If these files are missing, the app will run in **DEMO MODE** (simulated random predictions).

5.4 Step 3 — Run the Flask Web Application

```
# Make sure your virtual environment is activated
venv_final\Scripts\activate

# Run in production mode (default)
python web_dashboard_app.py

# OR run in development mode (enables debug, auto-reload)
set FLASK_ENV=development
python web_dashboard_app.py
```

You will see:

```
🔧 Loading models...
✅ Models loaded successfully!
🚀 CYBERBULLYING DETECTION SYSTEM (SECURE MODE + SHARED STORAGE)
* Running on http://0.0.0.0:5000
```

Access the application:

Page	URL
Main Analysis	http://localhost:5000
Dashboard	http://localhost:5000/dashboard
Explainable AI	http://localhost:5000/explain
YouTube Integration	http://localhost:5000/youtube
Health Check	http://localhost:5000/api/health

5.5 Step 4 — Load the Chrome Browser Extension

1. Open **Google Chrome**.
2. Go to `chrome://extensions/` in the address bar.
3. Enable **Developer Mode** (toggle in the top-right corner).
4. Click **"Load unpacked"**.
5. Select the folder: `D:\CyberbullyingDetection_FinalYear\browser_extension\`
6. The **ShieldGuard** icon will appear in your Chrome toolbar.
7. Make sure the Flask app is running (Step 3) — the extension popup will show **"System Online: AI Ready"** in green.

5.6 Using the REST API Directly

You can test the API directly using `curl` or Postman:

Predict text:

```
curl -X POST http://localhost:5000/api/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "You are so stupid and ugly"}'
```

Example Response:

```
{
  "is_cyberbullying": true,
  "label": "Other Cyberbullying",
  "confidence": 0.87,
  "stage1_safe": 0.12,
  "stage1_toxic": 0.88,
  "stage2_probabilities": { ... },
  "word_importance": { ... }
}
```

Health check:

```
curl http://localhost:5000/api/health
```

6. Common Issues & Troubleshooting

Issue	Cause	Fix
`Could not load models` warning	`best_model.bin` missing	Place trained `.bin` files in the `models/` folders
App runs in DEMO mode	Model files not found	Verify paths in `web_dashboard_app.py` lines 93–94
Extension shows "Offline"	Flask app not running	Start `web_dashboard_app.py` first
`ModuleNotFoundError`	Package not installed	Run `pip install -r requirements.txt` inside venv
Port 5000 already in use	Another process on port 5000	Change port: `app.run(port=5001)`
Extension not scanning	Page needs refresh after loading extension	Refresh the social media page after loading extension
`FileNotFoundError` in <code>evaluate_pipeline.py</code>	Wrong absolute paths	Update lines 32–34 with your actual paths